TrendFlix

تقرير شرح المشروع

شرح طبقة Controllers والواجهة View

تاريخ الإعداد: 2026-04-21

مقدمة

هذا التقرير يشرح مشروع **TrendFlix** من ناحية البنية البرمجية والمنطق الداخلي، مع التركيز الأساسي على **طبقة الـ Controllers** لأنها تمثل قلب المشروع: هي التي تستقبل الطلبات من الواجهة، وتتحقق من البيانات، وتطبق قواعد النظام، وتتواصل مع قاعدة البيانات، ثم تعيد النتيجة إلى المستخدم.

وبناءً على طلبك، أضفت أيضًا جزءًا واضحًا من **كود الواجهة View** سواء من ملفات HTML أو ملفات JavaScript الموجودة داخل `view/` و `view/assets/js/` ، حتى يكون التقرير شاملاً من جهة الباك إند والفرونت إند معًا.

المشروع مبني باستخدام:

- لغة **Go**
- إطار **Fiber**
- ORM باسم **GORM**
- MySQL
- واجهات HTML/CSS/JavaScript بدون إطار Frontend ثقيل

والمشروع يدعم الوظائف الآتية:

- تسجيل المستخدمين وتسجيل الدخول
- التحقق باستخدام JWT
- إعادة تعيين كلمة المرور بكود عبر البريد الإلكتروني
- إدارة العناصر من نوع: كتب، أفلام، ألعاب
- إدارة التصنيفات
- المفضلة
- المراجعات
- رفع الصور
- محادثة ذكاء اصطناعي مرتبطة بالأفلام والألعاب والكتب

1. الهيكل العام للمشروع

الهيكل الأساسي للمشروع مقسّم إلى طبقات واضحة:

الطبقة	الملفات الأساسية	الوظيفة
نقطة التشغيل	`main.go`	تشغيل التطبيق، فتح الاتصال بقاعدة البيانات، عمل migration وseed، ثم تسجيل المسارات
Routers	`routers/*.go`	ربط الـ URL بالـ Controller المناسب
Middleware	`middleware/auth_middleware.go`	التحقق من هوية المستخدم وصلاحيات الأدمن
Controllers	`controller/**`	منطق المشروع الرئيسي
Models	`models/*.go`	تعريف الجداول والعلاقات
Database	`database/*.go`	الاتصال بقاعدة البيانات والتهيئة الأولية
Utils	`utils/email.go`	خدمات مساعدة مثل إرسال البريد
View	`view/pages/**` و `view/assets/js/**`	صفحات الواجهة وسلوكها على المتصفح

بالتالي، سير الطلب داخل النظام يكون غالبًا كالتالي:

1. المستخدم يضغط زرًا أو يفتح صفحة في الواجهة.
2. JavaScript في الواجهة يرسل طلب HTTP.
3. الـ Router يحدد أي Controller يجب تشغيله.
4. الـ Middleware يفحص التوكن إذا كانت الصفحة أو العملية محمية.
5. الـ Controller يتحقق من البيانات ويطبق المنطق المطلوب.
6. قاعدة البيانات تُستخدم لحفظ أو جلب المعلومات.
7. النتيجة ترجع JSON إلى الواجهة.
8. الواجهة تعرض النتيجة للمستخدم.

2. بداية تشغيل التطبيق

أول ملف يجب فهمه هو `main.go` لأنه نقطة البداية الفعلية للمشروع.

مقطع كود: تشغيل التطبيق

الملف: `main.go`

```
func main() {
	database.ConnDB()
	database.Migrate()
	database.SeedAdmin()
	database.SeedCategories()
	database.SeedItems()

	app := fiber.New()
	app.Static("/upload", "./upload")

	routers.RegisterAuthRoutes(app)
	routers.RegisterChatRoutes(app)
	routers.RegisterCategoryRoutes(app)
	routers.RegisterFavoriteRoutes(app)
	routers.RegisterItemRoutes(app)
	routers.RegisterReviewRoutes(app)
	routers.RegisterUploadRoutes(app)
	routers.RegisterViewRoutes(app)

	port := os.Getenv("APP_PORT")
	if port == "" {
		port = "4000"
	}

	if err := app.Listen(":" + port); err != nil {
		panic(err)
	}
}
```

شرح هذا الجزء

- `ConnDB()` يفتح الاتصال بقاعدة البيانات.
- `Migrate()` ينشئ الجداول إن لم تكن موجودة.
- `SeedAdmin()` و `SeedCategories()` و `SeedItems()` يضيفون بيانات أولية حتى يصبح النظام جاهزًا للاستخدام.
- `app.Static("/upload", "./upload")` يجعل الصور المرفوعة قابلة للوصول من المتصفح.
- كل دالة من `Register...Routes` تضيف مجموعة endpoints للتطبيق.
- في النهاية يبدأ السيرفر على المنفذ المحدد.

معنى هذا أن التطبيق قبل أن يستقبل أي طلب يكون قد جهّز:

- قاعدة البيانات

- الجداول
- البيانات الافتراضية
- المسارات

3. كيف تصل الطلبات إلى الـ Controllers

الـ Router هو الطبقة التي تربط الرابط بالوظيفة المناسبة. مثال واضح جدًا على هذا هو راوتر العناصر.

مقطع كود: مسارات العناصر

الملف: `routers/item_router.go`

```
func RegisterItemRoutes(app *fiber.App) {
        items := app.Group("/items")
        items.Get("", itemcontroller.GetItems)
        items.Get("/:id", itemcontroller.GetItemByID)

        adminItems := app.Group("/items", middleware.Authenticate,
middleware.RequireAdmin)
        adminItems.Post("", itemcontroller.CreateItem)
        adminItems.Put("/:id", itemcontroller.UpdateItem)
        adminItems.Delete("/:id", itemcontroller.DeleteItem)
}
```

الشرح

- `GET /items` و `GET /items/:id` مسارات عامة، أي مستخدم يمكنه استعراض المحتوى.
- `POST /items` و `PUT /items/:id` و `DELETE /items/:id` مسارات محمية.
- قبل الوصول إلى هذه العمليات المحمية يتم تشغيل:
  - `Authenticate`
  - `RequireAdmin`

إذن الراوتر هنا لا ينفذ المنطق، لكنه يحدد:

- من يملك حق الوصول
- وأي Controller يجب تشغيله

4. الـ Middleware الخاص بالمصادقة والصلاحيات

هذا الملف مهم جدًا لأنه يحدد هل المستخدم مسموح له بالدخول أم لا.

مقطع كود: التحقق من JWT

الملف: `middleware/auth_middleware.go`

```
func Authenticate(c *fiber.Ctx) error {
	context := fiber.Map{
		"statusText": "bad",
		"msg":        "Unauthorized",
	}

	authorizationHeader := strings.TrimSpace(c.Get("Authorization"))
	if authorizationHeader == "" || !strings.HasPrefix(authorizationHeader,
"Bearer ") {
		return c.Status(fiber.StatusUnauthorized).JSON(context)
	}

	tokenString := strings.TrimSpace(strings.TrimPrefix(authorizationHeader,
"Bearer "))
	if tokenString == "" {
		return c.Status(fiber.StatusUnauthorized).JSON(context)
	}

	secret := strings.TrimSpace(os.Getenv("JWT_SECRET"))
	claims := jwt.MapClaims{}
	token, err := jwt.ParseWithClaims(tokenString, claims, func(token
*jwt.Token) (interface{}, error) {
		if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok {
			return nil, fmt.Errorf("unexpected signing method:
%v", token.Header["alg"])
		}

		return []byte(secret), nil
	})
	if err != nil || !token.Valid {
		return c.Status(fiber.StatusUnauthorized).JSON(context)
	}

	subject, ok := claims["sub"].(string)
	if !ok || strings.TrimSpace(subject) == "" {
		return c.Status(fiber.StatusUnauthorized).JSON(context)
	}

	userID, err := strconv.ParseUint(subject, 10, 64)
	if err != nil || userID == 0 {
		return c.Status(fiber.StatusUnauthorized).JSON(context)
	}

	var user models.User
```

```
        result := database.DbConn.First(&user, userID)
        if result.Error != nil {
                return c.Status(fiber.StatusUnauthorized).JSON(context)
        }

        c.Locals("currentUser", user)
        return c.Next()
}
```

الشرح

- يقرأ الهيدر `Authorization`.
- يتأكد أن قيمته تبدأ بـ `Bearer`.
- يستخرج التوكن.
- يتحقق من صحة التوقيع باستخدام `JWT_SECRET`.
- يقرأ `sub` من داخل الـ claims، وهو معرّف المستخدم.
- يجلب المستخدم من قاعدة البيانات.
- يخزن المستخدم داخل `c.Locals("currentUser", user)`.
- يسمح بإكمال الطلب باستخدام `c.Next()`.

هذا يعني أن أي Controller محمي لا يحتاج لإعادة فك التوكن مرة أخرى، بل يكفيه قراءة المستخدم الحالي من `Locals`.

مقطع كود: التحقق من صلاحية الأدمن

الملف: `middleware/auth_middleware.go`

```
func RequireAdmin(c *fiber.Ctx) error {
	context := fiber.Map{
		"statusText": "bad",
		"msg":        "Forbidden",
	}

	userValue := c.Locals("currentUser")
	user, ok := userValue.(models.User)
	if !ok {
		context["msg"] = "Unauthorized"
		return c.Status(fiber.StatusUnauthorized).JSON(context)
	}

	if strings.TrimSpace(user.Role) != "admin" {
		return c.Status(fiber.StatusForbidden).JSON(context)
	}

	return c.Next()
}
```

الشرح

- هذا الجزء لا يكتفي بأن المستخدم مسجّل دخول.
- بل يفرض أن `Role == "admin"` .
- لذلك أي عملية إدارة للعناصر أو التصنيفات لا يستطيع تنفيذها إلا الأدمن.

5. النمط العام المستخدم داخل الـ Controllers

أغلب الـ Controllers في المشروع تسير بنفس الترتيب تقريبًا:

1. إنشاء `context` أو `contextMap` لإرجاع الرسالة النهائية.
2. التأكد من وجود اتصال بقاعدة البيانات.
3. قراءة البيانات من `BodyParser` أو من الـ params أو من form-data.
4. تنظيف القيم باستخدام `TrimSpace` أو `ToLower` .
5. التحقق من صحة البيانات.
6. تنفيذ استعلامات قاعدة البيانات.
7. إرجاع النتيجة مع حالة HTTP مناسبة.

هذه الطريقة موحّدة في أغلب المشروع، وهذا يجعل الكود أسهل في الفهم والصيانة.

6. Controllers الخاصة بالمصادقة

ملفات المصادقة موجودة في:

- `controller/auth/auth_controller.go`
- `controller/auth/reset_password.go`

هذه الطبقة مسؤولة عن:

- إنشاء حساب
- تسجيل الدخول
- إنشاء JWT
- طلب إعادة تعيين كلمة المرور
- تنفيذ إعادة التعيين

6.1 إنشاء مستخدم جديد

مقطع كود: التسجيل

الملف: `controller/auth/auth_controller.go`

```
func CreateUser(c *fiber.Ctx) error {
	context := fiber.Map{
		"statusText": "Ok",
		"msg":        "Create User",
	}

	var request createUserRequest

	if err := c.BodyParser(&request); err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Invalid request"
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	request.Name = strings.TrimSpace(request.Name)
	request.Email = strings.ToLower(strings.TrimSpace(request.Email))
	request.Password = strings.TrimSpace(request.Password)
	request.Avatar = strings.TrimSpace(request.Avatar)

	if request.Name == "" || request.Email == "" || request.Password == "" {
		context["statusText"] = "bad"
		context["msg"] = "Name, email and password are required"
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	var existingUser models.User
	result := database.DbConn.Where("email = ?",
request.Email).First(&existingUser)
	if result.RowsAffected > 0 {
		context["statusText"] = "bad"
		context["msg"] = "Email already exists"
		return c.Status(fiber.StatusConflict).JSON(context)
	}

	hashedPassword, err :=
bcrypt.GenerateFromPassword([]byte(request.Password), bcrypt.DefaultCost)
	if err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Error hashing password"
		return c.Status(fiber.StatusInternalServerError).JSON(context)
	}

	record := models.User{
		Name:     request.Name,
```

```
                Email:    request.Email,
                Password: string(hashedPassword),
                Avatar:   request.Avatar,
                Role:     "user",
        }

        result = database.DbConn.Create(&record)
        if result.Error != nil {
                context["statusText"] = "bad"
                context["msg"] = "Error in saving user"
                return c.Status(fiber.StatusInternalServerError).JSON(context)
        }

        token, err := generateAuthToken(record)
        if err != nil {
                context["statusText"] = "bad"
                context["msg"] = "Error generating token"
                return c.Status(fiber.StatusInternalServerError).JSON(context)
        }

        context["id"] = record.ID
        context["token"] = token
        context["msg"] = "User created successfully"
        return c.Status(fiber.StatusCreated).JSON(context)
}
```

الشرح

- يقرأ البيانات القادمة من نموذج التسجيل.
- ينظف الاسم والبريد وكلمة المرور.
- يرفض الطلب إذا كانت القيم الأساسية ناقصة.
- يتأكد أن البريد غير مكرر.
- يحوّل كلمة المرور إلى hash باستخدام `bcrypt` .
- ينشئ المستخدم داخل قاعدة البيانات.
- ينشئ توكن JWT مباشرة بعد التسجيل.
- يعيد للمستخدم:
  - `id`
  - `token`

هذه نقطة مهمة جدًا لأن النظام يتيح للمستخدم أن ينتقل مباشرة من التسجيل إلى داخل التطبيق دون تسجيل دخول جديد.

6.2 تسجيل الدخول وإنشاء JWT

مقطع كود: تسجيل الدخول

الملف: `controller/auth/auth_controller.go`

```
func LoginUser(c *fiber.Ctx) error {
	context := fiber.Map{
		"statusText": "Ok",
		"msg":        "Login Successful",
	}

	var request loginRequest
	if err := c.BodyParser(&request); err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Invalid request"
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	request.Email = strings.ToLower(strings.TrimSpace(request.Email))
	request.Password = strings.TrimSpace(request.Password)

	if request.Email == "" || request.Password == "" {
		context["statusText"] = "bad"
		context["msg"] = "Email and password are required"
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	var user models.User
	result := database.DbConn.Where("email = ?", request.Email).First(&user)
	if result.Error != nil {
		context["statusText"] = "bad"
		context["msg"] = "Invalid email or password"
		return c.Status(fiber.StatusUnauthorized).JSON(context)
	}

	err := bcrypt.CompareHashAndPassword([]byte(user.Password),
[]byte(request.Password))
	if errors.Is(err, bcrypt.ErrMismatchedHashAndPassword) {
		context["statusText"] = "bad"
		context["msg"] = "Invalid email or password"
		return c.Status(fiber.StatusUnauthorized).JSON(context)
	}

	token, err := generateAuthToken(user)
	if err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Authentication error"
		return c.Status(fiber.StatusInternalServerError).JSON(context)
	}
```

```
        context["id"] = user.ID
        context["role"] = user.Role
        context["token"] = token
        return c.Status(fiber.StatusOK).JSON(context)
}
```

مقطع كود: إنشاء التوكن

الملف: `controller/auth/auth_controller.go`

```
func generateAuthToken(user models.User) (string, error) {
        secret := strings.TrimSpace(os.Getenv("JWT_SECRET"))
        now := time.Now()
        claims := jwt.MapClaims{
                "sub":  strconv.FormatUint(uint64(user.ID), 10),
                "role": user.Role,
                "iat":  now.Unix(),
                "exp":  now.Add(24 * time.Hour).Unix(),
        }

        token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
        return token.SignedString([]byte(secret))
}
```

الشرح

- `LoginUser` يبحث عن المستخدم بالبريد الإلكتروني.
- يقارن كلمة المرور المدخلة بالـ hash المخزّن.
- إذا نجح التحقق ينشئ JWT جديدًا.
- التوكن يحتوي على:
  - `sub` : رقم المستخدم
  - `role` : دور المستخدم
  - `iat` : وقت الإنشاء
  - `exp` : وقت انتهاء الصلاحية

وجود `role` داخل التوكن مهم جدًا لأن الواجهة تستخدمه أيضًا لإظهار بعض عناصر الأدمن، بينما الباك إند يستخدمه للحماية الفعلية.

6.3 إعادة تعيين كلمة المرور

تدفق إعادة تعيين كلمة المرور مكوّن من مرحلتين:

1. إرسال كود إلى البريد
2. إدخال الكود وتغيير كلمة المرور

مقطع كود: طلب كود إعادة التعيين

الملف: `controller/auth/reset_password.go`

```
func ResetPasswordRequest(c *fiber.Ctx) error {
	context := fiber.Map{
		"statusText": "Ok",
		"msg":        "If the email exists, a reset code has been
sent",
	}

	var request resetPasswordEmailRequest
	if err := c.BodyParser(&request); err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Invalid request body"
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	request.Email = strings.ToLower(strings.TrimSpace(request.Email))
	if request.Email == "" {
		context["statusText"] = "bad"
		context["msg"] = "Email is required"
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	var user models.User
	result := database.DbConn.Where("email = ?", request.Email).First(&user)
	if result.Error != nil {
		if errors.Is(result.Error, gorm.ErrRecordNotFound) {
			return c.Status(fiber.StatusOK).JSON(context)
		}
		return c.Status(fiber.StatusInternalServerError).JSON(context)
	}

	resetCode, err := generateResetCode()
	if err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Error generating reset code"
		return c.Status(fiber.StatusInternalServerError).JSON(context)
	}

	expiresAt := time.Now().Add(resetTokenDuration())
	err = database.DbConn.Transaction(func(tx *gorm.DB) error {
		if err := tx.Where("user_id = ?",
user.ID).Delete(&models.ResetToken{}).Error; err != nil {
			return err
		}
```

```
                resetToken := models.ResetToken{
                        UserID:    user.ID,
                        Code:      resetCode,
                        ExpiresAt: expiresAt,
                }

                return tx.Create(&resetToken).Error
        })
        if err != nil {
                context["statusText"] = "bad"
                context["msg"] = "Error saving reset token"
                return c.Status(fiber.StatusInternalServerError).JSON(context)
        }

        if err := utils.SendEmail(user.Email, "Password Reset Code", resetCode);
err != nil {
                database.DbConn.Where("user_id = ? AND code = ?", user.ID,
resetCode).Delete(&models.ResetToken{})
                context["statusText"] = "bad"
                context["msg"] = "Error sending email"
                return c.Status(fiber.StatusInternalServerError).JSON(context)
        }

        return c.Status(fiber.StatusOK).JSON(context)
}
```

الشرح

- لا يعلن للمستخدم بشكل صريح إن كان البريد موجودًا أم لا، وهذا أفضل أمنيًا.
- ينشئ كودًا عشوائيًا من 6 أرقام.
- يحذف أي كود قديم للمستخدم.
- يحفظ كودًا جديدًا مع وقت انتهاء صلاحية.
- يرسل الكود بالبريد.
- لو فشل الإرسال يحذف الكود من قاعدة البيانات حتى لا يبقى كود غير مستخدم.

إذن هذا الـ Controller لا ينفذ مجرد حفظ بسيط، بل يدير تدفقًا أمنيًا كاملًا.

7. Controller العناصر Item Controller

هذا الـ Controller هو أهم جزء في المشروع تقريبًا، لأنه يتعامل مع الكيان الأساسي في النظام: **العنصر**.

العنصر قد يكون:

- فيلم
- لعبة
- كتاب

لذلك هذا الملف يحتوي على منطق أعمال حقيقي، وليس فقط CRUD بسيط.

7.1 نموذج العنصر

مقطع كود: نموذج Item

الملف: `models/item_model.go`

```
type Item struct {
        ID          uint       `gorm:"column:id;primaryKey;autoIncrement"
json:"id"`
        Title       string     `gorm:"column:title;not null" json:"title"`
        Description string     `gorm:"column:description;type:text"
json:"description"`
        Type        string     `gorm:"column:type;not null" json:"type"`
        CoverImage  string     `gorm:"column:cover_image" json:"cover_image"`
        ContentLink *string    `gorm:"column:content_link" json:"content_link"`
        ReleaseDate time.Time  `gorm:"column:release_date;type:date"
json:"release_date"`
        Author      *string    `gorm:"column:author" json:"author"`
        Director    *string    `gorm:"column:director" json:"director"`
        Developer   *string    `gorm:"column:developer" json:"developer"`
        Duration    *uint      `gorm:"column:duration" json:"duration"`
        PagesCount  *uint      `gorm:"column:pages_count" json:"pages_count"`
        Platform    *string    `gorm:"column:platform" json:"platform"`
        Rating      float64    `gorm:"column:rating" json:"rating"`
        Categories  []Category
`gorm:"many2many:category_item;joinForeignKey:ItemID;joinReferences:CategoryID"
json:"categories,omitempty"`
}
```

الشرح

- ليس كل نوع يحتاج نفس الحقول.
- الكتاب يحتاج مثلًا:
  - `Author`
  - `PagesCount`
- الفيلم يحتاج:

  - `Director`
  - `Duration`
- اللعبة تحتاج:
  - `Developer`
  - `Platform`

ولهذا السبب يحتوي `item_controller.go` على منطق يزيل الحقول غير المناسبة لكل نوع.

7.2 إنشاء عنصر جديد

مقطع كود: إنشاء عنصر

الملف: `controller/item_controller/item_controller.go`

```
func CreateItem(c *fiber.Ctx) error {
	context := fiber.Map{
		"statusText": "Ok",
		"msg":        "Item created successfully",
	}

	if err := requireAdminAccess(c, context); err != nil {
		return err
	}

	var request itemRequest
	if err := c.BodyParser(&request); err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Invalid request"
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	record, msg, statusCode := buildItemFromRequest(request)
	if statusCode != 0 {
		context["statusText"] = "bad"
		context["msg"] = msg
		return c.Status(statusCode).JSON(context)
	}

	categories, msg, statusCode := loadCategoriesByIDs(request.CategoryIDs)
	if statusCode != 0 {
		context["statusText"] = "bad"
		context["msg"] = msg
		return c.Status(statusCode).JSON(context)
	}

	tx := database.DbConn.Begin()
	if tx.Error != nil {
		context["statusText"] = "bad"
		context["msg"] = "Database error"
		return c.Status(fiber.StatusInternalServerError).JSON(context)
	}

	if err := tx.Create(&record).Error; err != nil {
		tx.Rollback()
		context["statusText"] = "bad"
		context["msg"] = "Error saving item"
		return c.Status(fiber.StatusInternalServerError).JSON(context)
	}
```

```
        if err := replaceItemCategories(tx, &record, categories); err != nil {
                tx.Rollback()
                context["statusText"] = "bad"
                context["msg"] = "Error saving item categories"
                return c.Status(fiber.StatusInternalServerError).JSON(context)
        }

        if err := tx.Commit().Error; err != nil {
                context["statusText"] = "bad"
                context["msg"] = "Database error"
                return c.Status(fiber.StatusInternalServerError).JSON(context)
        }

        record.Categories = categories

        context["id"] = record.ID
        context["item"] = record
        return c.Status(fiber.StatusCreated).JSON(context)
}
```

الشرح

- أول خطوة هي التأكد أن المستخدم أدمن.
- ثم يتم قراءة البيانات القادمة من الواجهة.
- `buildItemFromRequest` ينفذ التحقق الأساسي على الحقول.
- `loadCategoriesByIDs` يتأكد أن التصنيفات المطلوبة موجودة فعلًا.
- يبدأ transaction لأن العملية تتكوّن من خطوتين مترابطتين:
  - إنشاء العنصر
  - ربط العنصر بالتصنيفات
- لو فشلت أي خطوة يتم عمل rollback.
- لو نجحت العمليتان يتم عمل commit.

هذا منطق مهم جدًا لأنه يحافظ على **اتساق البيانات**.

7.3 التحقق من نوع العنصر والحقول المناسبة

مقطع كود: بناء العنصر من الطلب

الملف: `controller/item_controller/item_controller.go`

```
func buildItemFromRequest(request itemRequest) (models.Item, string, int) {
	request.Title = strings.TrimSpace(request.Title)
	request.Description = strings.TrimSpace(request.Description)
	request.Type = strings.ToLower(strings.TrimSpace(request.Type))
	request.CoverImage = strings.TrimSpace(request.CoverImage)
	request.ContentLink = normalizeOptionalString(request.ContentLink)
	request.ReleaseDate = strings.TrimSpace(request.ReleaseDate)
	request.Author = normalizeOptionalString(request.Author)
	request.Director = normalizeOptionalString(request.Director)
	request.Developer = normalizeOptionalString(request.Developer)
	request.Platform = normalizeOptionalString(request.Platform)

	if request.Title == "" || request.Type == "" || request.ReleaseDate ==
"" {
		return models.Item{}, "Title, type and release_date are
required", fiber.StatusBadRequest
	}

	if request.Type != "book" && request.Type != "movie" && request.Type !=
"game" {
		return models.Item{}, "Type must be book, movie or game",
fiber.StatusBadRequest
	}

	releaseDate, err := time.Parse("2006-01-02", request.ReleaseDate)
	if err != nil {
		return models.Item{}, "release_date must be in YYYY-MM-DD
format", fiber.StatusBadRequest
	}

	item := models.Item{
		Title:       request.Title,
		Description: request.Description,
		Type:        request.Type,
		CoverImage:  request.CoverImage,
		ContentLink: request.ContentLink,
		ReleaseDate: releaseDate,
		Author:      request.Author,
		Director:    request.Director,
		Developer:   request.Developer,
		Duration:    request.Duration,
		PagesCount:  request.PagesCount,
		Platform:    request.Platform,
		Rating:      request.Rating,
```

```
        }

        switch item.Type {
        case "book":
                item.Director = nil
                item.Developer = nil
                item.Duration = nil
                item.Platform = nil
        case "movie":
                item.Author = nil
                item.Developer = nil
                item.PagesCount = nil
                item.Platform = nil
        case "game":
                item.Author = nil
                item.Director = nil
                item.Duration = nil
                item.PagesCount = nil
        }

        return item, "", 0
}
```

الشرح

- ينظف كل الحقول النصية.
- يفرض أن النوع واحد من القيم:
  - `book`
  - `movie`
  - `game`
- يتحقق من أن التاريخ بصيغة صحيحة.
- ينشئ كائن `Item` .
- ثم يطبق منطقًا مهمًا جدًا:
  - إذا كان العنصر كتابًا، تُحذف خصائص الفيلم واللعبة.
  - إذا كان فيلمًا، تُحذف خصائص الكتاب واللعبة.
  - إذا كان لعبة، تُحذف خصائص الكتاب والفيلم.

هذا هو أحد أقوى أمثلة **المنطق البرمجي الحقيقي** داخل المشروع.

7.4 تحميل التصنيفات وربط العلاقة

مقطع كود: تحميل التصنيفات

الملف: `controller/item_controller/item_controller.go`

```
func loadCategoriesByIDs(categoryIDs []uint) ([]models.Category, string, int) {
	if len(categoryIDs) == 0 {
		return []models.Category{}, "", 0
	}

	uniqueIDs := make([]uint, 0, len(categoryIDs))
	seen := make(map[uint]struct{}, len(categoryIDs))
	for _, categoryID := range categoryIDs {
		if categoryID == 0 {
			return nil, "Invalid category id",
fiber.StatusBadRequest
		}

		if _, exists := seen[categoryID]; exists {
			continue
		}

		seen[categoryID] = struct{}{}
		uniqueIDs = append(uniqueIDs, categoryID)
	}

	var categories []models.Category
	result := database.DbConn.Where("id IN ?", uniqueIDs).Find(&categories)
	if result.Error != nil {
		return nil, "Database error", fiber.StatusInternalServerError
	}

	categoryByID := make(map[uint]models.Category, len(categories))
	for _, category := range categories {
		categoryByID[category.ID] = category
	}

	orderedCategories := make([]models.Category, 0, len(uniqueIDs))
	for _, categoryID := range uniqueIDs {
		category, exists := categoryByID[categoryID]
		if !exists {
			return nil, "One or more categories were not found",
fiber.StatusBadRequest
		}

		orderedCategories = append(orderedCategories, category)
	}

	return orderedCategories, "", 0
```

```
}

func replaceItemCategories(tx *gorm.DB, item *models.Item, categories
[]models.Category) error {
	association := tx.Model(item).Association("Categories")
	if association.Error != nil {
		return association.Error
	}

	if len(categories) == 0 {
		return association.Clear()
	}

	return association.Replace(categories)
}
```

الشرح

- يحذف التكرارات من `category_ids` .
- يمنع القيم `0` .
- يتأكد أن كل تصنيف مطلوب موجود فعليًا.
- يعيد ترتيب التصنيفات حسب الترتيب القادم من الطلب.
- يربط العلاقة many-to-many بين `items` و `categories` .

إذن الـ Controller هنا لا يتعامل فقط مع جدول واحد، بل مع:

- جدول العناصر
- جدول التصنيفات
- جدول الربط `category_item`

7.5 التحديث والحذف

دالتي `UpdateItem` و `DeleteItem` مبنيتان بنفس الأسلوب:

- التحقق من أن المستخدم أدمن
- التحقق من الـ id
- جلب العنصر من قاعدة البيانات
- تنفيذ التعديل أو الحذف داخل transaction

والسبب في استخدام transaction هنا هو منع حدوث حالة يكون فيها:

- العنصر تم تعديله
- لكن التصنيفات لم تتحدث

أو العكس.

8. Controller المراجعات Reviews

هذا الجزء يسمح للمستخدمين بإضافة مراجعات على العناصر. الملفات الأساسية هنا هي:

- `controller/reviews_controller/reviews_controller.go`

وهو مسؤول عن:

- عرض مراجعات عنصر معين
- إنشاء مراجعة
- تعديل المراجعة
- حذف المراجعة

8.1 إنشاء مراجعة

مقطع كود: إنشاء مراجعة

الملف: `controller/reviews_controller/reviews_controller.go`

```
func CreateReview(c *fiber.Ctx) error {
	context := fiber.Map{
		"statusText": "Ok",
		"msg":        "Review created successfully",
	}

	user, err := currentUserFromContext(c, context)
	if err != nil {
		return err
	}

	var request createReviewRequest
	if err := c.BodyParser(&request); err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Invalid request"
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	review, msg, statusCode := buildReviewFromCreateRequest(request)
	if statusCode != 0 {
		context["statusText"] = "bad"
		context["msg"] = msg
		return c.Status(statusCode).JSON(context)
	}

	if err := ensureItemExists(review.ItemID, context, c); err != nil {
		return err
	}

	var existingReview models.Review
	result := database.DbConn.Where("user_id = ? AND item_id = ?", user.ID,
review.ItemID).First(&existingReview)
	if result.Error == nil {
		context["statusText"] = "bad"
		context["msg"] = "You already reviewed this item"
		return c.Status(fiber.StatusConflict).JSON(context)
	}

	review.UserID = user.ID

	if err := database.DbConn.Create(&review).Error; err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Error saving review"
		return c.Status(fiber.StatusInternalServerError).JSON(context)
```

```
        }

        context["id"] = review.ID
        context["review"] = review
        return c.Status(fiber.StatusCreated).JSON(context)
}
```

الشرح

- يأخذ المستخدم الحالي من الـ middleware.
- يقرأ التقييم والتعليق ومعرّف العنصر.
- يتأكد من أن العنصر موجود.
- يمنع المستخدم من إضافة أكثر من مراجعة لنفس العنصر.
- يفرض ملكية المراجعة باستخدام `review.UserID = user.ID`.
- يحفظ المراجعة.

إذن منطق هذا الـ Controller ليس فقط الحفظ، بل أيضًا:

- حماية الملكية
- منع التكرار
- التحقق من وجود العنصر

8.2 التحقق من قيمة التقييم

مقطع كود: التحقق من التقييم

الملف: `controller/reviews_controller/reviews_controller.go`

```
func normalizeReviewInput(rating uint, comment string) (uint, string, string,
int) {
        comment = strings.TrimSpace(comment)

        if rating < 1 || rating > 5 {
                return 0, "", "Rating must be between 1 and 5",
fiber.StatusBadRequest
        }

        return rating, comment, "", 0
}
```

الشرح

- المشروع يفرض أن التقييم من 1 إلى 5.

- يتم تنظيف التعليق من الفراغات الزائدة.
- نفس الدالة تُستخدم في create وupdate.

هذه إعادة استخدام جيدة للمنطق بدل تكراره في أكثر من مكان.

9. Controller المفضلة Favorites

هذا الجزء يسمح للمستخدم بحفظ العناصر التي يحبها في قائمة خاصة به.

9.1 جلب المفضلة

مقطع كود: جلب قائمة المفضلة

الملف: `controller/favorites_controller/favorites_controller.go`

```
func GetFavorites(c *fiber.Ctx) error {
	context := fiber.Map{
		"statusText": "Ok",
		"msg":        "Favorites fetched successfully",
	}

	user, err := currentUserFromContext(c, context)
	if err != nil {
		return err
	}

	var items []models.Item
	result := database.DbConn.
		Model(&models.Item{}).
		Joins("JOIN favorites ON favorites.item_id = items.id").
		Where("favorites.user_id = ?", user.ID).
		Preload("Categories").
		Order("favorites.created_at DESC").
		Find(&items)
	if result.Error != nil {
		context["statusText"] = "bad"
		context["msg"] = "Database error"
		return c.Status(fiber.StatusInternalServerError).JSON(context)
	}

	context["items"] = items
	return c.Status(fiber.StatusOK).JSON(context)
}
```

الشرح

- هذا الـ Controller لا يعيد فقط سجلات جدول `favorites` .
- بل ينفذ `JOIN` مع جدول `items` .
- وبالتالي الواجهة تحصل على بيانات العنصر نفسه، لا مجرد المعرفات.
- كما أنه يحمّل التصنيفات أيضًا باستخدام `Preload("Categories")` .

هذه نقطة مهمة لأنها تجعل استجابة الـ API جاهزة للعرض مباشرة في الواجهة.

9.2 إضافة عنصر إلى المفضلة

مقطع كود: إضافة للمفضلة

الملف: `controller/favorites_controller/favorites_controller.go`

```
func AddFavorite(c *fiber.Ctx) error {
	context := fiber.Map{
		"statusText": "Ok",
		"msg":        "Favorite added successfully",
	}

	user, err := currentUserFromContext(c, context)
	if err != nil {
		return err
	}

	itemID, err := parseItemID(c, context)
	if err != nil {
		return err
	}

	if err := ensureItemExists(itemID, context, c); err != nil {
		return err
	}

	var favorite models.Favorite
	result := database.DbConn.Where("user_id = ? AND item_id = ?", user.ID,
itemID).First(&favorite)
	if result.Error == nil {
		context["statusText"] = "bad"
		context["msg"] = "Item already in favorites"
		return c.Status(fiber.StatusConflict).JSON(context)
	}

	favorite = models.Favorite{
		UserID: user.ID,
		ItemID: itemID,
	}

	if err := database.DbConn.Create(&favorite).Error; err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Error saving favorite"
		return c.Status(fiber.StatusInternalServerError).JSON(context)
	}

	context["id"] = favorite.ID
	context["favorite"] = favorite
```

```
        return c.Status(fiber.StatusCreated).JSON(context)
}
```

الشرح

- يقرأ `item_id` من الرابط.
- يتأكد أن العنصر موجود.
- يتأكد أن المستخدم لم يضفه سابقًا.
- يحفظ العلاقة بين المستخدم والعنصر.

إذن وظيفة هذا الـ Controller الأساسية هي حماية نظافة العلاقة بين:

- `user`
- `item`

10. Controller التصنيفات Categories

هذا الملف مسؤول عن:

- عرض التصنيفات
- إنشاء تصنيف
- تعديل تصنيف
- حذف تصنيف

10.1 إنشاء تصنيف

مقطع كود: إنشاء تصنيف

الملف: `controller/categories_controller/categories_controller.go`

```
func CreateCategory(c *fiber.Ctx) error {
	context := fiber.Map{
		"statusText": "Ok",
		"msg":        "Category created successfully",
	}

	var request categoryRequest
	if err := c.BodyParser(&request); err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Invalid request"
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	request.Name = strings.TrimSpace(request.Name)
	request.Slug = strings.TrimSpace(request.Slug)

	if request.Name == "" || request.Slug == "" {
		context["statusText"] = "bad"
		context["msg"] = "Name and slug are required"
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	var existingCategory models.Category
	result := database.DbConn.Where("slug = ?",
request.Slug).First(&existingCategory)
	if result.RowsAffected > 0 {
		context["statusText"] = "bad"
		context["msg"] = "Slug already exists"
		return c.Status(fiber.StatusConflict).JSON(context)
	}

	record := models.Category{
		Name: request.Name,
		Slug: request.Slug,
	}

	result = database.DbConn.Create(&record)
	if result.Error != nil {
		context["statusText"] = "bad"
		context["msg"] = "Error saving category"
		return c.Status(fiber.StatusInternalServerError).JSON(context)
	}

	context["id"] = record.ID
```

```
        context["category"] = record
        return c.Status(fiber.StatusCreated).JSON(context)
}
```

الشرح

- يتأكد أن الاسم والـ slug موجودان.
- يمنع تكرار الـ slug.
- ينشئ التصنيف داخل قاعدة البيانات.

وباقي الدوال في الملف تطبق نفس الفكرة على التعديل والحذف والاسترجاع.

11. Controller رفع الصور Upload

هذا الـ Controller صغير نسبيًا لكنه مهم عمليًا لأن لوحة الأدمن تعتمد عليه عند رفع صورة عنصر جديد.

مقطع كود: منطق رفع الصورة

الملف: `controller/upload_controller/upload_controller.go`

```
func UploadAvatar(c *fiber.Ctx) error {
	return uploadImage(c, "avatars", "Avatar uploaded successfully")
}

func UploadItemImage(c *fiber.Ctx) error {
	return uploadImage(c, "items", "Item image uploaded successfully")
}

func uploadImage(c *fiber.Ctx, subDir string, successMessage string) error {
	context := fiber.Map{
		"statusText": "Ok",
		"msg":        successMessage,
	}

	fileHeader, err := c.FormFile("file")
	if err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Image file is required"
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	if err := validateImageFile(fileHeader); err != nil {
		context["statusText"] = "bad"
		context["msg"] = err.Error()
		return c.Status(fiber.StatusBadRequest).JSON(context)
	}

	storageDir := filepath.Join(uploadRootDir, subDir)
	if err := os.MkdirAll(storageDir, 0o755); err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Error preparing upload directory"
		return c.Status(fiber.StatusInternalServerError).JSON(context)
	}

	fileName := buildUploadFileName(fileHeader.Filename)
	storagePath := filepath.Join(storageDir, fileName)
	if err := c.SaveFile(fileHeader, storagePath); err != nil {
		context["statusText"] = "bad"
		context["msg"] = "Error saving image"
		return c.Status(fiber.StatusInternalServerError).JSON(context)
	}

	publicPath := "/upload/" + subDir + "/" + fileName
	context["path"] = publicPath
```

```
        context["file_name"] = fileName
        return c.Status(fiber.StatusCreated).JSON(context)
}
```

الشرح

- يقبل الملف من `multipart/form-data`.
- يتحقق من امتداد الصورة ونوعها.
- ينشئ المجلد إذا لم يكن موجودًا.
- يحفظ الملف فعليًا على القرص.
- يرجع المسار العام للصورة.

هذا المسار تستخدمه الواجهة لاحقًا عند حفظ العنصر داخل `/items`.

12. Controller المحادثة Chat

هذا الملف يضيف بعدًا ذكيًا للمشروع لأنه يربط التطبيق بخدمة OpenRouter.

12.1 إرسال الطلب إلى خدمة الذكاء الاصطناعي

مقطع كود: تجهيز طلب المحادثة

الملف: `controller/chat_controller/chat_controller.go`

```
func Reply(c *fiber.Ctx) error {
	contextMap := fiber.Map{
		"statusText": "Ok",
		"msg":        "Chat reply generated successfully",
	}

	apiKey := strings.TrimSpace(os.Getenv("OPENROUTER_API_KEY"))
	if apiKey == "" {
		contextMap["statusText"] = "bad"
		contextMap["msg"] = "Chat service is unavailable right now"
		return
c.Status(fiber.StatusServiceUnavailable).JSON(contextMap)
	}

	var request chatRequest
	if err := c.BodyParser(&request); err != nil {
		contextMap["statusText"] = "bad"
		contextMap["msg"] = "Invalid request"
		return c.Status(fiber.StatusBadRequest).JSON(contextMap)
	}

	message := strings.TrimSpace(request.Message)
	if message == "" {
		contextMap["statusText"] = "bad"
		contextMap["msg"] = "Message is required"
		return c.Status(fiber.StatusBadRequest).JSON(contextMap)
	}

	if len([]rune(message)) > maxMessageLength {
		contextMap["statusText"] = "bad"
		contextMap["msg"] = "Message is too long"
		return c.Status(fiber.StatusBadRequest).JSON(contextMap)
	}

	messages := []openRouterEntry{
		{Role: "system", Content: trendFlixSystemPrompt},
		{Role: "system", Content: buildLanguageInstruction(message)},
	}
	messages = append(messages, normalizeHistory(request.History)...)
	messages = append(messages, openRouterEntry{Role: "user", Content:
message})

	payload := openRouterRequest{
		Model:       getOpenRouterModel(),
```

```
                Messages:    messages,
                Temperature: 0.4,
                MaxTokens:   280,
                TopP:        0.9,
        }
```

الشرح

- يأخذ الرسالة من المستخدم.
- يرفض الرسائل الفارغة أو الطويلة جدًا.
- يضيف `system prompt` ثابت يحدد أن TrendFlix لا يجيب إلا على أسئلة الكتب والأفلام والألعاب.
- يضيف تعليمات اللغة حسب كون الرسالة عربية أو إنجليزية.
- يضيف history للمحادثة لكن بعد تنظيفها.
- يرسل الطلب إلى OpenRouter.

12.2 تنظيف الـ history وتحديد اللغة

مقطع كود: تنقية الرسائل السابقة

الملف: `controller/chat_controller/chat_controller.go`

```
func normalizeHistory(history []openRouterEntry) []openRouterEntry {
	if len(history) == 0 {
		return nil
	}

	start := 0
	if len(history) > maxChatHistoryMessages {
		start = len(history) - maxChatHistoryMessages
	}

	normalized := make([]openRouterEntry, 0, len(history)-start)
	for _, entry := range history[start:] {
		role := strings.TrimSpace(strings.ToLower(entry.Role))
		if role != "user" && role != "assistant" {
			continue
		}

		content := strings.TrimSpace(entry.Content)
		if content == "" {
			continue
		}

		runes := []rune(content)
		if len(runes) > maxMessageLength {
			content = string(runes[:maxMessageLength])
		}

		normalized = append(normalized, openRouterEntry{Role: role,
Content: content})
	}

	return normalized
}
```

الشرح

- لا يستخدم كامل سجل المحادثة، بل آخر عدد محدد فقط.
- يقبل فقط رسائل:
  - `user`
  - `assistant`
- يحذف الرسائل الفارغة.

- يقص الرسائل الطويلة.

هذا يحافظ على:

- سرعة الطلب
- وضوح الـ prompt
- تقليل الاستهلاك غير الضروري

13. لماذا الـ Controllers هي قلب المشروع

في هذا المشروع لا توجد طبقة services منفصلة بين الـ controllers وقاعدة البيانات، ولذلك الـ controllers تقوم فعليًا بعدة أدوار معًا:

- Request handlers
- Validation layer
- Business logic layer
- Response formatter

ولهذا السبب عندما نريد شرح المشروع أكاديميًا، فإن أفضل مكان للتركيز هو طبقة الـ Controllers.

14. الواجهة View: لماذا يجب شرحها أيضًا

بما أن المشروع ليس API فقط، فالواجهة مهمة جدًا لفهم كيف يصل المستخدم إلى الـ Controllers عمليًا.

الواجهة هنا مقسمة إلى جزأين:

- صفحات HTML داخل `view/pages/`
- منطق JavaScript داخل `view/assets/js/`

فكرة التصميم في هذا المشروع هي:

- HTML يوفّر الهيكل العام للصفحة
- JavaScript يجلب البيانات من الـ API
- ثم يبني العناصر ديناميكيًا ويعرضها للمستخدم

وهذا يجعل الربط بين الفرونت إند والباك إند واضحًا ومباشرًا.

15. واجهة تسجيل الدخول

15.1 صفحة HTML لتسجيل الدخول

مقطع كود: صفحة تسجيل الدخول

الملف: `view/pages/auth/auth.html`

```
<main class="card auth-card">
  <h1 class="brand">TrendFlix</h1>
  <p class="subtitle" data-i18n="auth.loginSubtitle">Sign in to continue</p>

  <form id="loginForm" class="form">
    <label class="field">
      <span data-i18n="common.email">Email</span>
      <input name="email" type="email" autocomplete="email" required />
    </label>
    <label class="field password-field">
      <span data-i18n="common.password">Password</span>
      <input name="password" type="password" autocomplete="current-password"
required />
      <button type="button" class="password-toggle" data-i18n-aria-
label="common.togglePasswordVisibility">
        ...
      </button>
    </label>

    <button class="btn primary" type="submit" data-
i18n="auth.login">Login</button>
    <p id="errorMsg" class="error" hidden></p>
  </form>
</main>
```

الشرح

- الصفحة تحتوي نموذجًا بسيطًا فيه:
  - البريد الإلكتروني
  - كلمة المرور
  - زر تسجيل الدخول
- يوجد عنصر `errorMsg` لعرض رسالة الخطأ لو فشل الطلب.
- يوجد أيضًا `data-i18n` مما يدل على أن الصفحة تدعم الترجمة.

إذن HTML هنا يحدد الشكل العام للنموذج، لكن لا ينفذ عملية الدخول نفسها.

15.2 JavaScript تسجيل الدخول

مقطع كود: استدعاء API تسجيل الدخول

الملف: `view/assets/js/auth.js`

```
async function login(email, password) {
  const res = await fetch("/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });

  const data = await res.json().catch(() => ({}));
  if (!res.ok) {
    const msg = data?.msg || t("auth.loginFailed");
    throw new Error(msg);
  }

  if (!data?.token) {
    throw new Error(t("auth.noToken"));
  }
  return data.token;
}

window.addEventListener("DOMContentLoaded", () => {
  const form = document.getElementById("loginForm");
  form.addEventListener("submit", async (e) => {
    e.preventDefault();

    const fd = new FormData(form);
    const email = String(fd.get("email") || "").trim();
    const password = String(fd.get("password") || "").trim();

    const tokenValue = await login(email, password);
    localStorage.setItem(TOKEN_KEY, tokenValue);
    window.location.replace("/pages/app.html");
  });
});
```

الشرح

- عند إرسال النموذج، يمنع JavaScript الإرسال التقليدي للصفحة.
- يجمع البيانات من النموذج.
- يرسل `POST /auth/login` .
- إذا وصل التوكن بنجاح، يحفظه في `localStorage` .

- ثم ينقل المستخدم إلى الصفحة الرئيسية `/pages/app.html` .

هذه نقطة الربط المباشر بين:

- الواجهة
- و `LoginUser` داخل الـ controller

أي أن الفرونت إند هنا يعتمد على الـ controller اعتمادًا مباشرًا وواضحًا.

16. الصفحة الرئيسية للتطبيق

16.1 الهيكل العام للواجهة

مقطع كود: الصفحة الرئيسية

الملف: `view/pages/app.html`

```
<aside class="sidebar" id="sidebar" data-i18n-aria-label="app.sidebarNavigation">
  <a href="#" data-nav="home">🏠 <span data-i18n="app.home">Home</span></a>
  <a href="#" data-nav="library">📚 <span data-i18n="app.library">Library</span>
</a>
  <a href="/pages/favorites.html" data-nav="favorites">❤️ <span data-
i18n="app.favorites">Favorites</span></a>
  <a href="/pages/admin.html" id="adminNavLink" hidden>🛠️ <span data-
i18n="app.adminDashboard">Admin Dashboard</span></a>
  <button class="link danger" id="logoutBtn" type="button">🚪 <span data-
i18n="app.logout">Logout</span></button>
</aside>

<main id="catalogSections" class="catalog-sections" aria-live="polite">
  <p class="catalog-status" id="catalogStatus" data-
i18n="app.loadingCatalog">Loading catalog...</p>
</main>

<button class="ai-icon" id="aiToggle" type="button" data-i18n-aria-
label="app.openAiChat">🤖</button>
<section class="chat-box" id="chatBox" data-i18n-aria-label="app.aiChat" aria-
hidden="true">
  <div id="chatLogs" class="chat-logs">
    <div class="msg bot-msg" data-i18n="app.chatWelcome">Welcome to TrendFlix. I
only answer questions about movies, games, and books.</div>
  </div>
  <form class="chat-input-area" id="chatForm">
    <input type="text" id="userInput" placeholder="Ask about movies, games, or
books..." />
    <button id="chatSubmit" type="submit">➤</button>
  </form>
</section>
```

الشرح

- الصفحة الرئيسية تعرض:
  - القائمة الجانبية
  - منطقة عرض الكتالوج
  - زر فتح المحادثة الذكية
- العنصر `adminNavLink` مخفي افتراضيًا.
- `catalogSections` هو المكان الذي سيملؤه JavaScript ببطاقات العناصر.
- `chatBox` هو واجهة المحادثة مع الذكاء الاصطناعي.

إذن HTML هنا يضع الحاويات الأساسية فقط، بينما البيانات تُملأ لاحقًا من الـ API.

16.2 تحميل العناصر والتصنيفات

مقطع كود: تحميل الكتالوج من الواجهة

الملف: `view/assets/js/app.js`

```
async function fetchJson(url, options = {}, token = "") {
  const headers = {
    Accept: "application/json",
    ...(options.headers || {}),
  };
  if (token) headers.Authorization = `Bearer ${token}`;

  const response = await fetch(url, {
    ...options,
    headers,
  });

  const data = await response.json().catch(() => ({}));
  if (!response.ok) {
    throw new Error(data?.msg || `Request failed: ${response.status}`);
  }

  return data;
}

async function loadCatalog() {
  setCatalogStatus("app.loadingCatalog");

  const [itemsResponse, categoriesResponse] = await
Promise.all([fetchJson("/items"), fetchJson("/categories")]);

  items = Array.isArray(itemsResponse?.items) ? itemsResponse.items : [];
  categories = Array.isArray(categoriesResponse?.categories) ?
categoriesResponse.categories : [];

  renderCatalog();
}
```

الشرح

- هذه الدالة تنادي:

  - `/items`
  - `/categories`
- وتحمّل البيانات بالتوازي باستخدام `Promise.all`.
- بعد ذلك تستدعي `renderCatalog()` لعرض الأقسام داخل الصفحة.

وهنا نرى الربط المباشر بين:

- `GetItems` في الباك إند
- `GetCategories` في الباك إند
- وعرض النتائج في الصفحة الرئيسية

16.3 إظهار أو إخفاء لوحة الأدمن من الواجهة

مقطع كود: قراءة role من JWT

الملف: `view/assets/js/app.js`

```
function parseJwtPayload(token) {
  try {
    const payload = token.split(".")[1] || "";
    const normalized = payload.replaceAll("-", "+").replaceAll("_", "/");
    const padded = normalized.padEnd(Math.ceil(normalized.length / 4) * 4, "=");
    return JSON.parse(window.atob(padded));
  } catch {
    return null;
  }
}

function getCurrentRole() {
  const token = localStorage.getItem(TOKEN_KEY);
  if (!token) return "";

  const payload = parseJwtPayload(token);
  return String(payload?.role || "").trim().toLowerCase();
}

function syncAdminNavLink() {
  const adminNavLink = document.getElementById("adminNavLink");
  if (!adminNavLink) return;

  adminNavLink.hidden = getCurrentRole() !== "admin";
}
```

الشرح

- الواجهة تقرأ `role` من داخل JWT.
- إذا كان المستخدم `admin` يظهر رابط لوحة التحكم.
- إذا لم يكن أدمن يبقى الرابط مخفيًا.

مهم جدًا ملاحظة أن هذا **تحكم واجهي فقط** وليس حماية أمنية حقيقية. الحماية الفعلية موجودة في الباك إند داخل `RequireAdmin`.

16.4 التبديل بين المفضلة داخل الصفحة الرئيسية

مقطع كود: إضافة أو إزالة من المفضلة

الملف: `view/assets/js/app.js`

```
async function toggleFavorite(btn) {
  const itemId = btn.getAttribute("data-item-id") || "";
  if (!itemId || !currentToken) return;

  const isActive = btn.classList.contains("active");
  btn.disabled = true;

  try {
    await fetchJson(`/favorites/${itemId}`, { method: isActive ? "DELETE" :
"POST" }, currentToken);
    btn.classList.toggle("active", !isActive);
    if (isActive) favoriteItemIds.delete(itemId);
    else favoriteItemIds.add(itemId);
  } catch (error) {
    console.error("Failed to toggle favorite", error);
  } finally {
    btn.disabled = false;
  }
}
```

الشرح

- الزر نفسه يقرر هل العملية:
  - إضافة
  - أم حذف
- لو كان العنصر مفعّلًا في المفضلة يرسل `DELETE`.
- لو لم يكن مفعّلًا يرسل `POST`.

- بعد نجاح الطلب يغيّر شكل الزر في الصفحة.

هذا يربط مباشرة مع:

- `AddFavorite`
- `RemoveFavorite`

17. صفحة التفاصيل Detail Page

17.1 ملف HTML الخاص بالتفاصيل

مقطع كود: صفحة التفاصيل

الملف: `view/pages/detail.html`

```
<body>
  <div id="detailRoot">
    <div class="detail-loading" id="detailLoading">
      <div class="spinner"></div>
      <p data-i18n="detail.loading">Loading...</p>
    </div>
  </div>
</body>
```

الشرح

- هذه الصفحة تحتوي هيكلًا بسيطًا جدًا.
- يوجد فقط `detailRoot` .
- JavaScript هو من يبني الصفحة كاملة داخله.

إذن الصفحة هنا تعتمد على **rendering ديناميكي**.

17.2 تحميل العنصر والمراجعات

مقطع كود: جلب بيانات صفحة التفاصيل

الملف: `view/assets/js/detail.js`

```
window.addEventListener("DOMContentLoaded", async () => {
  const token = requireAuth();
  if (!token) return;
  currentToken = token;

  const id = getItemIdFromLocation();
  if (!id) {
    window.location.replace("/pages/app.html");
    return;
  }

  const root = document.getElementById("detailRoot");

  try {
    const [itemRes, reviewsRes] = await Promise.all([
      fetchJson(`/items/${id}`, token),
      fetchJson(`/reviews/item/${id}`, token).catch(() => ({ reviews: [] })),
    ]);

    const item = itemRes?.item || null;
    if (!item?.id) {
      root.innerHTML = `
        <div class="detail-error">
          <p>${escapeHtml(t("detail.notFound"))}</p>
          <a href="/pages/app.html" class="detail-error-back">←
${escapeHtml(t("detail.back"))}</a>
        </div>`;
      return;
    }

    currentItem = item;
    currentReviews = Array.isArray(reviewsRes?.reviews) ? reviewsRes.reviews :
[];

    document.title = `TrendFlix · ${item.title || "Details"}`;

    root.innerHTML = buildPage(item, currentReviews);
    window.TrendFlixI18n?.translatePage();
    attachHandlers(item, currentReviews, token);
  } catch (err) {
    root.innerHTML = `
      <div class="detail-error">
        <p>${escapeHtml(t("detail.loadFailed"))}</p>
        <a href="/pages/app.html" class="detail-error-back">←
```

```
${escapeHtml(t("detail.back"))}</a>
      </div>`;
  }
});
```

الشرح

- عند فتح الصفحة، يتأكد أولًا من وجود توكن.
- يقرأ `id` من الرابط.
- يجلب بالتوازي:
  - بيانات العنصر من `/items/:id`
  - مراجعات العنصر من `/reviews/item/:id`
- إذا لم يجد العنصر يعرض رسالة خطأ مناسبة.
- إذا نجح، يبني الصفحة ديناميكيًا.

وهذا مثال ممتاز على التكامل بين الفرونت إند والباك إند.

17.3 بناء واجهة التفاصيل وربط الأزرار

مقطع كود: إنشاء الصفحة وتفعيل الأزرار

الملف: `view/assets/js/detail.js`

```
function buildPage(item, reviews) {
  const meta = getTypeMeta(item.type);
  const safeImg = item.cover_image || getFallbackImage(item.title);
  const safeTitle = escapeHtml(item.title || "");
  const contentLink = String(item.content_link || "").trim();
  const metaHtml = buildMeta(item);
  const cats = buildCategories(item.categories);
  const reviewsHtml = buildReviews(reviews);

  return `
    <div class="detail-page">
      <div class="detail-hero">
        <div class="detail-poster">
          <img src="${escapeHtml(safeImg)}" alt="${safeTitle}" />
        </div>

        <div class="detail-info">
          <h1 class="detail-title">${safeTitle}</h1>
          ${cats ? `<div class="detail-categories">${cats}</div>` : ""}
          ${metaHtml ? `<div class="detail-meta">${metaHtml}</div>` : ""}

          <div class="detail-actions">
            <button class="action-primary" id="actionBtn" type="button"
${contentLink ? `data-content-link="${escapeHtml(contentLink)}"` : "disabled"}>
              ${meta.actionIcon} ${escapeHtml(t(meta.actionKey))}
            </button>
            <button class="action-fav" id="favBtn" type="button" data-item-
id="${item.id}">
              ♥ <span id="favBtnLabel">${escapeHtml(t("detail.addFavorite"))}
</span>
            </button>
          </div>
        </div>
      </div>

      <section class="detail-section">
        <h2 class="section-title">${escapeHtml(t("detail.reviews"))}</h2>
        <div class="reviews-grid">${reviewsHtml}</div>
      </section>
    </div>
  `;
}

function attachHandlers(item, reviews, token) {
```

```
  document.getElementById("actionBtn")?.addEventListener("click", () => {
    const contentLink = String(item.content_link || "").trim();
    if (!contentLink) return;
    window.open(contentLink, "_blank", "noopener,noreferrer");
  });

  const favBtn = document.getElementById("favBtn");
  if (favBtn) {
    favBtn.addEventListener("click", async () => {
      const isActive = favBtn.classList.contains("active");
      const id = favBtn.getAttribute("data-item-id");
      if (!id) return;

      const method = isActive ? "DELETE" : "POST";
      await fetch(`/favorites/${id}`, {
        method,
        headers: { Authorization: `Bearer ${token}`, Accept: "application/json"
},
      });
    });
  }
}
```

الشرح

- `buildPage` تحول بيانات JSON القادمة من الـ API إلى HTML فعلي.
- الزر الأساسي `actionBtn` يفتح رابط المشاهدة أو القراءة أو اللعب.
- زر `favBtn` يربط صفحة التفاصيل مباشرة مع Controller المفضلة.
- كما أن الصفحة تعرض:
  - التصنيفات
  - التفاصيل الخاصة بالنوع
  - المراجعات

18. صفحة المفضلة Favorites View

18.1 واجهة عرض المفضلة

مقطع كود: ملف JavaScript للمفضلة

الملف: `view/assets/js/favorites.js`

```
async function loadFavorites(token) {
  setStatus("favorites.loading");
  const response = await fetchJson("/favorites", token);
  favoriteItems = Array.isArray(response?.items) ? response.items : [];
  renderFavorites();
}

async function removeFavorite(itemId, token) {
  await fetchJson(`/favorites/${itemId}`, token, { method: "DELETE" });
  favoriteItems = favoriteItems.filter((item) => String(item.id) !==
String(itemId));
  renderFavorites();
}
```

الشرح

- الصفحة تنادي `GET /favorites` لتحميل العناصر المحفوظة.
- وعند الحذف تنادي `DELETE /favorites/:itemId` .
- بعد نجاح الحذف يتم تحديث المصفوفة داخل الواجهة ثم إعادة الرسم مباشرة.

وهذا يجعل تجربة الاستخدام سريعة دون الحاجة إلى إعادة تحميل الصفحة كاملة.

19. واجهة لوحة التحكم Admin لإنشاء العناصر

هذا الجزء مهم جدًا لأنه يشرح كيف تتصل لوحة التحكم مباشرة بأهم Controllers في المشروع.

19.1 صفحة إنشاء عنصر

مقطع كود: نموذج إنشاء عنصر

الملف: `view/pages/admin/create-item.html`

```
<form id="itemForm" class="form-stack">
  <div class="form-grid two-col">
    <label class="field">
      <span data-i18n="admin.itemName">Title</span>
      <input name="title" type="text" required />
    </label>
    <label class="field">
      <span data-i18n="admin.itemType">Type</span>
      <select id="itemType" name="type">
        <option value="movie">Movie</option>
        <option value="game">Game</option>
        <option value="book">Book</option>
      </select>
    </label>
  </div>

  <div class="form-grid image-grid">
    <label class="field grow-field">
      <span data-i18n="admin.coverImage">Cover image URL</span>
      <input id="coverImageInput" name="cover_image" type="url" />
    </label>
    <label class="field">
      <span data-i18n="admin.uploadImage">Upload image</span>
      <input id="coverImageFile" name="cover_upload" type="file" accept="image/*"
/>
    </label>
  </div>

  <div class="form-grid two-col" id="typeSpecificFields">
    <label class="field" data-field-for="book">
      <span data-i18n="admin.author">Author</span>
      <input name="author" type="text" />
    </label>
    <label class="field" data-field-for="movie">
      <span data-i18n="admin.director">Director</span>
      <input name="director" type="text" />
    </label>
    <label class="field" data-field-for="game">
      <span data-i18n="admin.developer">Developer</span>
      <input name="developer" type="text" />
    </label>
  </div>

  <div id="itemCategoryList" class="checkbox-list"></div>
```

```
  <button class="btn primary" id="itemSubmitBtn" type="submit">Create
item</button>
</form>
```

الشرح

- النموذج يحتوي على الحقول العامة لكل عنصر.
- كما يحتوي حقولًا متخصصة حسب نوع العنصر.
- توجد قائمة تصنيفات يتم ملؤها ديناميكيًا.
- يوجد خياران للصورة:
  - رابط مباشر
  - رفع ملف

هذه الصفحة تعتمد بشكل مباشر على JavaScript كي تضبط الحقول الصحيحة لكل نوع.

19.2 تفعيل الحقول حسب النوع وتجهيز البيانات

مقطع كود: تفعيل حقول النوع

الملف: `view/assets/js/admin-item-form.js`

```
function syncTypeFields() {
  const type = document.getElementById("itemType")?.value || "movie";
  document.querySelectorAll("[data-field-for]").forEach((field) => {
    const visible = field.getAttribute("data-field-for") === type;
    field.classList.toggle("is-hidden", !visible);
    field.querySelectorAll("input, select, textarea").forEach((input) => {
      input.disabled = !visible;
    });
  });
}

function buildItemPayload(form) {
  const formData = new FormData(form);
  const type = String(formData.get("type") || "").trim();
  const payload = {
    title: String(formData.get("title") || "").trim(),
    description: String(formData.get("description") || "").trim(),
    type,
    cover_image: String(formData.get("cover_image") || "").trim(),
    content_link: String(formData.get("content_link") || "").trim() || null,
    release_date: String(formData.get("release_date") || "").trim(),
    rating: Number(formData.get("rating") || 0),
    category_ids: getSelectedCategoryIds(),
  };

  if (type === "book") {
    payload.author = String(formData.get("author") || "").trim() || null;
    payload.pages_count = parseOptionalInteger(formData.get("pages_count"));
  }
  if (type === "movie") {
    payload.director = String(formData.get("director") || "").trim() || null;
    payload.duration = parseOptionalInteger(formData.get("duration"));
  }
  if (type === "game") {
    payload.developer = String(formData.get("developer") || "").trim() || null;
    payload.platform = String(formData.get("platform") || "").trim() || null;
  }

  return payload;
}
```

الشرح

- `syncTypeFields()` يظهر فقط الحقول المناسبة للنوع المختار.
- `buildItemPayload()` يبني JSON النهائي الذي سيُرسل للباك إند.
- نفس الفكرة الموجودة في الواجهة تتوافق مع منطق `buildItemFromRequest()` في الباك إند.

وهذا تطابق ممتاز بين الفرونت إند والباك إند:

- الواجهة تظهر الحقول المناسبة
- والباك إند يفرض القواعد نفسها مرة أخرى

19.3 رفع الصورة ثم إنشاء العنصر

مقطع كود: رفع الصورة ثم إرسال العنصر

الملف: `view/assets/js/admin-item-form.js`

```
async function uploadCoverImageIfNeeded() {
  const fileInput = document.getElementById("coverImageFile");
  const coverInput = document.getElementById("coverImageInput");
  const file = fileInput?.files?.[0];

  if (!file) return String(coverInput?.value || "").trim();

  const uploadFile = await compressImageFile(file, 0.75);

  const formData = new FormData();
  formData.append("file", uploadFile);

  const response = await fetchJson("/upload/item-image", {
    method: "POST",
    headers: authHeaders(),
    body: formData,
  });

  const path = String(response?.path || "").trim();
  if (coverInput) coverInput.value = path;
  updateImagePreview(path);
  return path;
}

async function handleItemSubmit(event) {
  event.preventDefault();

  const mode = getFormMode();
  const itemId = getItemIdFromUrl();
  const isEditMode = mode === "edit";

  await uploadCoverImageIfNeeded();
  const payload = buildItemPayload(event.target);

  const response = await fetchJson(isEditMode ? `/items/${itemId}` : "/items", {
    method: isEditMode ? "PUT" : "POST",
    headers: authHeaders({ "Content-Type": "application/json" }),
    body: JSON.stringify(payload),
  });
}
```

الشرح

- إذا اختار الأدمن ملف صورة، يتم رفعه أولًا إلى:
  - `/upload/item-image`
- ثم يأخذ المسار الناتج ويضعه داخل `cover_image`.
- بعد ذلك يرسل بيانات العنصر إلى:
  - `POST /items` عند الإنشاء
  - أو `PUT /items/:id` عند التعديل

هذا يوضح تسلسلًا عمليًا جميلًا في المشروع:

1. رفع الصورة
2. حفظ رابط الصورة
3. إنشاء العنصر نفسه

20. واجهة الكتالوج في لوحة الأدمن

مقطع كود: تحميل قائمة العناصر في لوحة التحكم

الملف: `view/assets/js/admin-catalog.js`

```
async function loadItems() {
  clearNotice("pageError");
  const data = await fetchJson("/items");
  items = Array.isArray(data?.items) ? data.items : [];
  render();
}

window.addEventListener("DOMContentLoaded", async () => {
  if (!requireAdmin()) return;

  document.getElementById("refreshItemsBtn")?.addEventListener("click", () =>
    loadItems().catch(showPageError),
  );

  document.getElementById("catalogFilters")?.addEventListener("click", (event) =>
{
    const btn = event.target.closest("[data-filter]");
    if (!btn) return;
    activeFilter = btn.dataset.filter;
    render();
  });

  try {
    await loadItems();
  } catch (error) {
    showPageError(error);
  }
});
```

الشرح

- لوحة الأدمن تعيد استخدام endpoint عام هو `/items` .
- لكنها تضيف فوقه:
  - فلاتر حسب النوع
  - أزرار تحرير
  - إعادة تحميل

هذا يعني أن نفس API يمكن أن يخدم:

- الصفحة العامة للمستخدم
- ولوحة الإدارة

لكن كل واجهة تعرض البيانات بالطريقة المناسبة لها.

21. كيف تتكامل الواجهة مع الـ Controllers

أفضل طريقة لفهم المشروع هي تتبع سيناريوهات حقيقية:

21.1 سيناريو تسجيل الدخول

1. المستخدم يملأ نموذج `auth.html` .
2. `auth.js` يرسل `POST /auth/login` .
3. `LoginUser` يتحقق من البريد وكلمة المرور.
4. يرجع JWT.
5. `auth.js` يحفظ التوكن في `localStorage` .
6. يتم تحويل المستخدم إلى `/pages/app.html` .

21.2 سيناريو تحميل الصفحة الرئيسية

1. `app.js` يتحقق من وجود التوكن.
2. ينادي:
   - `/items`
   - `/categories`
3. `GetItems` و `GetCategories` يعيدان البيانات.
4. `renderCatalog()` يبني الأقسام والبطاقات.
5. إذا كان المستخدم أدمن، يظهر رابط لوحة الأدمن.

21.3 سيناريو فتح صفحة التفاصيل

1. المستخدم يضغط على بطاقة عنصر.
2. `app.js` يوجهه إلى `/pages/detail.html?id=...` .
3. `detail.js` يجلب:
   - `/items/:id`
   - `/reviews/item/:id`
4. الصفحة تُبنى ديناميكيًا.
5. يمكنه بعدها:
   - فتح رابط المشاهدة أو القراءة أو اللعب
   - أو الإضافة إلى المفضلة

21.4 سيناريو إنشاء عنصر من لوحة الأدمن

1. الأدمن يفتح `create-item.html` .
2. `admin-item-form.js` يحمّل التصنيفات من `/categories` .
3. الأدمن يملأ النموذج.
4. إذا رفع صورة، يرسلها إلى `/upload/item-image` .
5. بعد ذلك يرسل JSON إلى `/items` .
6. `CreateItem` يتحقق من البيانات ويحفظ العنصر والتصنيفات.
7. الواجهة تعرض رسالة نجاح.

22. لماذا هذا المشروع جيد كمشروع تخرج

هذا المشروع يحتوي على عناصر قوية جدًا تجعله مناسبًا كمشروع تخرج:

- نظام مصادقة كامل
- صلاحيات مستخدم/أدمن
- CRUD حقيقي على كيان رئيسي
- علاقات many-to-many
- رفع ملفات
- مراجعات ومفضلة
- تكامل مع بريد إلكتروني
- تكامل مع خدمة ذكاء اصطناعي خارجية
- واجهة مترابطة مع الباك إند بشكل مباشر

أي أنه ليس مشروعًا شكليًا فقط، بل يحتوي على:

- منطق أعمال
- حماية وصلاحيات
- تكاملات خارجية
- تجربة مستخدم كاملة

23. الخلاصة

إذا أردنا تلخيص المشروع باختصار أكاديمي:

- الـ **Controllers** هي الطبقة الأهم لأنها تطبق منطق المشروع الحقيقي.
- الـ **Middleware** يضمن أن الوصول إلى العمليات المحمية آمن.
- الـ **Models** تمثل الجداول والعلاقات.

- **الواجهة View** لا تعمل وحدها، بل تعتمد بالكامل على الـ API الذي تبنيه الـ Controllers.

أهم Controller في المشروع هو **Item Controller** لأنه يدير الكيان الرئيسي ويحتوي على أقوى قواعد التحقق والتعامل مع التصنيفات والأنواع المختلفة.

أما من جهة الواجهة، فأهم جزء هو أن ملفات JavaScript لا تعرض بيانات ثابتة، بل تعتمد على:

- استدعاء الـ API
- استقبال JSON
- بناء الصفحة ديناميكيًا
- وتحديث الواجهة حسب حالة المستخدم وصلاحياته

وبهذا يصبح TrendFlix مشروعًا متكاملًا يربط بين:

- الباك إند
- قاعدة البيانات
- الواجهة
- الذكاء الاصطناعي

في تطبيق واحد واضح البنية وسهل الشرح في مشروع التخرج.